

■ 难解性

实际上我们不仅需要确定, 算法对任何输入都能够在有穷次操作之后终止, 而且更加关注该过程所需的时间。很遗憾, 很多算法即便满足有穷性, 但在终止之前所花费的时间成本却太高。比如, 理论研究的成果显示, 大量问题的最低求解时间成本, 都远远超出目前实际系统所能提供的计算能力。同样地, 此类难解性 (intractability) 问题, 在本书中也不予过多讨论。

■ 计算效率

在“编写合法程序”这一基础之上, 本书将更多地关注于非“不可解和难解”的一般性问题, 并讨论如何高效率地解决这一层面的计算问题。为此, 首先需要确立一种尺度, 用以从时间和空间等方面度量算法的计算成本, 进而依此尺度对不同算法进行比较和评判。当然, 更重要的是研究和归纳算法设计与实现过程中的一般性规律与技巧, 以编写出效率更高、能够处理更大规模数据的程序。这两点既是本书的基本主题, 也是贯穿始终的主体脉络。

■ 数据结构

由上可知, 无论是算法的初始输入、中间结果还是最终输出, 在计算机中都可以数据的形式表示。对于数据的存储、组织、转移及变换等操作, 不同计算模型和平台环境所支持的具体形式不尽相同, 其执行效率将直接影响和决定算法的整体效率。数据结构这一学科正是以“数据”这一信息的表现形式为研究对象, 旨在建立支持高效算法的数据信息处理策略、技巧与方法。要做到根据实际应用需求自如地设计、实现和选用适当的数据结构, 必须首先对算法设计的技巧以及相应数据结构的特性了然于心, 这些也是本书的重点与难点。

§ 1.2 复杂度度量

算法的计算成本涵盖诸多方面, 为确定计算成本的度量标准, 我们不妨先从计算速度这一主要因素入手。具体地, 如何度量一个算法所需的计算时间呢?

1.2.1 时间复杂度

上述问题并不容易直接回答, 原因在于, 运行时间是由多种因素综合作用而决定的。首先, 即使是同一算法, 对于不同的输入所需的运行时间并不相同。以排序问题为例, 输入序列的规模、其中各元素的数值以及次序均不确定, 这些因素都将影响到排序算法最终的运行时间。为针对运行时间建立起一种可行、可信的评估标准, 我们不得不首先考虑其中最为关键的因素。其中, 问题实例的规模往往是决定计算成本的主要因素。一般地, 问题规模越接近, 相应的计算成本也越接近; 而随着问题规模的扩大, 计算成本通常也呈上升趋势。

如此, 本节开头所提的问题即可转化为: 随着输入规模的扩大, 算法的执行时间将如何增长? 执行时间的这一变化趋势可表示为输入规模的一个函数, 称作该算法的时间复杂度 (time complexity)。具体地, 特定算法处理规模为 n 的问题所需的时间可记作 $T(n)$ 。

细心的读者可能注意到, 根据规模并不能唯一确定具体的输入, 规模相同的输入通常都有多个, 而算法对其进行处理所需时间也不尽相同。仍以排序问题为例, 由 n 个元素组成的输入序列有 $n!$ 种, 有时所有元素都需交换, 有时却无需任何交换 (习题[1-3])。故严格说来, 以上定义的 $T(n)$ 并不明确。为此需要再做一次简化, 即从保守估计的角度出发, 在规模为 n 的所有输入中选择执行时间最长者作为 $T(n)$, 并以 $T(n)$ 度量该算法的时间复杂度。

1.2.2 渐进复杂度

至此，对于同一问题的两个算法A和B，通过比较其时间复杂度 $T_A(n)$ 和 $T_B(n)$ ，即可评价二者对于同一输入规模 n 的计算效率高低。然而，藉此还不足以就其性能优劣做出总体性的评判，比如对于某些问题，一些算法更适用于小规模输入，而另一些则相反（习题[1-5]）。

幸运的是，在评价算法运行效率时，我们往往可以忽略其处理小规模问题时的能力差异，转而关注其在处理更大规模问题时的表现。其中的原因不难理解，小规模问题所需的处理时间本来就相对更少，故此时不同算法的实际效率差异并不明显；而在处理更大规模的问题时，效率的些许差异都将对实际执行效果产生巨大的影响。这种着眼长远、更为注重时间复杂度的总体变化趋势和增长速度的策略与方法，即所谓的渐进分析（asymptotic analysis）。

那么，针对足够大的输入规模 n ，算法执行时间 $T(n)$ 的渐进增长速度，应如何度量和评价呢？

■ 大O记号

同样地出于保守的估计，我们首先关注 $T(n)$ 的渐进上界。为此可引入所谓“大O记号”（big-O notation）。具体地，若存在正的常数 c 和函数 $f(n)$ ，使得对任何 $n \gg 2$ 都有

$$T(n) \leq c \cdot f(n)$$

则可认为在 n 足够大之后， $f(n)$ 给出了 $T(n)$ 增长速度的一个渐进上界。此时，记之为：

$$T(n) = O(f(n))$$

由这一定义，可导出大O记号的以下性质：

- (1) 对于任一常数 $c > 0$ ，有 $O(f(n)) = O(c \cdot f(n))$
- (2) 对于任意常数 $a > b > 0$ ，有 $O(n^a + n^b) = O(n^a)$

前一性质意味着，在大O记号的意义下，函数各项正的常系数可以忽略并等同于1。后一性质则意味着，多项式中的低次项均可忽略，只需保留最高次项。可以看出，大O记号的这些性质的确体现了对函数总体渐进增长趋势的关注和刻画。

■ 环境差异

在实际环境中直接测得的执行时间 $T(n)$ ，虽不失为衡量算法性能的一种指标，但作为评判不同算法性能优劣的标准，其可信度值得推敲。事实上，即便是同一算法、同一输入，在不同的硬件平台上、不同的操作系统中甚至不同的时间，所需要的计算时间都不尽相同。因此，有必要按照超脱于具体硬件平台和软件环境的某一客观标准，来度量算法的时间复杂度，并进而评价不同算法的效率差异。

■ 基本操作

一种自然且可行的解决办法是，将时间复杂度理解为算法中各条指令的执行时间之和。在图灵机（Turing Machine, TM）和随机存储机（Random Access Machine, RAM）等计算模型^[4]中，指令语句均可分解为若干次基本操作，比如算术运算、比较、分支、子程序调用与返回等；而在大多数实际的计算环境中，每一次这类基本操作都可在常数时间内完成。

如此，不妨将 $T(n)$ 定义为算法所执行基本操作的总次数。也就是说， $T(n)$ 决定于组成算法的所有语句各自的执行次数，以及其中所含基本操作的数目。以代码1.1中起泡排序bubblesort1A()算法为例，若将该算法处理长度为 n 的序列所需的时间记作 $T(n)$ ，则按照上述分析，只需统计出该算法所执行基本操作的总次数，即可确定 $T(n)$ 的上界。

■ 起泡排序

`bubblesort1A()`算法由内、外两层循环组成。内循环从前向后,依次比较各对相邻元素,如有必要则将其交换。故在每一轮内循环中,需要扫描和比较 $n - 1$ 对元素,至多需要交换 $n - 1$ 对元素。元素的比较和交换,都属于基本操作,故每一轮内循环至多需要执行 $2(n - 1)$ 次基本操作。另外,根据1.1.4节对该算法正确性的分析结论,外循环至多执行 $n - 1$ 轮。因此,总共需要执行的基本操作不会超过 $2(n - 1)^2$ 次。若以此来度量该算法的时间复杂度,则有

$$T(n) = O(2(n-1)^2)$$

根据大 O 记号的性质,可进一步简化和整理为:

$$T(n) = O(2n^2 - 4n + 2) = O(2n^2) = O(n^2)$$

■ 最坏、最好与平均情况

由上可见,以大 O 记号形式表示的时间复杂度,实质上是对算法执行时间的一种保守估计——对于规模为 n 的任意输入,算法的运行时间都不会超过 $O(f(n))$ 。比如,“起泡排序算法复杂度 $T(n) = O(n^2)$ ”意味着,该算法处理任何序列所需的时间绝不会超过 $O(n^2)$ 。的确需要这么长计算时间的输入实例,称作最坏实例或最坏情况(**worst case**)。

需强调的是,这种保守估计并不排斥更好情况甚至最好情况(**best case**)的存在和出现。比如,对于某些输入序列,起泡排序算法的内循环的执行轮数可能少于 $n - 1$,甚至只需执行一轮(习题[1-3])。当然,有时也需要考查所谓的平均情况(**average case**),也就是按照某种约定的概率分布,将规模为 n 的所有输入对应的计算时间加权平均。

比较而言,“最坏情况复杂度”是人们最为关注且使用最多的,在一些特殊的场合甚至成为唯一的指标。比如控制核电站运转、管理神经外科手术室现场的系统而言,从最好或平均角度评判算法的响应速度都不具有任何意义,在最坏情况下的响应速度才是唯一的指标。

■ 大 Ω 记号

为了对算法的复杂度最好情况做出估计,需要借助另一个记号。如果存在正的常数 c 和函数 $g(n)$,使得对于任何 $n \gg 2$ 都有

$$T(n) \geq c \cdot g(n)$$

就可以认为,在 n 足够大之后, $g(n)$ 给出了 $T(n)$ 的一个渐进下界。此时,我们记之为:

$$T(n) = \Omega(g(n))$$

这里的 Ω 称作“大 Ω 记号”(big-omega notation)。与大 O 记号恰好相反,大 Ω 记号是对算法执行效率的乐观估计——对于规模为 n 的任意输入,算法的运行时间都不低于 $\Omega(g(n))$ 。比如,即便在最好情况下,起泡排序也至少需要 $T(n) = \Omega(n)$ 的计算时间(习题[1-4])。

■ 大 Θ 记号

借助大 O 记号、大 Ω 记号,可以对算法的时间复杂度作出定量的界定,亦即,从渐进的趋势看, $T(n)$ 介于 $\Omega(g(n))$ 与 $O(f(n))$ 之间。若恰巧出现 $g(n) = f(n)$ 的情况,则可以使用另一记号来表示。

如果存在正的常数 $c_1 < c_2$ 和函数 $h(n)$,使得对于任何 $n \gg 2$ 都有

$$c_1 \cdot h(n) \leq T(n) \leq c_2 \cdot h(n)$$

就可以认为在 n 足够大之后, $h(n)$ 给出了 $T(n)$ 的一个确界。此时,我们记之为:

$$T(n) = \Theta(h(n))$$

这里的 Θ 称作“大 Θ 记号”（big-theta notation），它是对算法复杂度的准确估计——对于规模为 n 的任何输入，算法的运行时间 $T(n)$ 都与 $\Theta(h(n))$ 同阶。

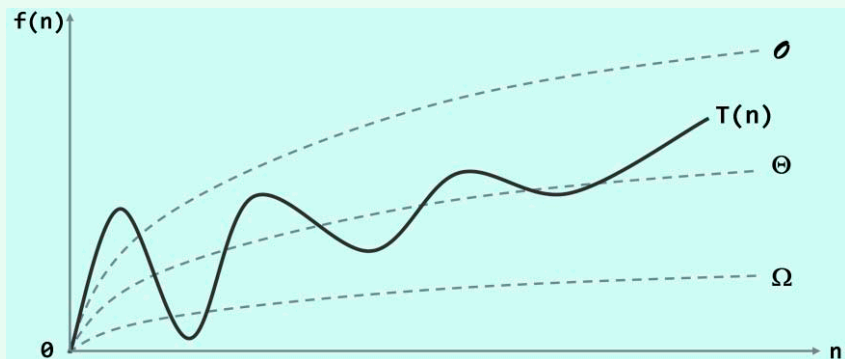


图1.4 大 O 记号、大 Ω 记号和大 Θ 记号

以上主要的这三种渐进复杂度记号之间的联系与区别，可直观地由图1.4示意。

1.2.3 空间复杂度

除了执行时间的长短，算法所需存储空间的多少也是衡量其性能的一个重要方面，此即所谓的空间复杂度（space complexity）。实际上，以上针对时间复杂度所引入的几种渐进记号，也适用于对空间复杂度的度量，其原理及方法基本相同，不再赘述。

需要注意的是，为了更为客观地评价算法性能的优劣，除非特别申明，空间复杂度通常并不计入原始输入本身所占用的空间——对于同一问题，这一指标对任何算法都是相同的。反之，其它（如转储、中转、索引、映射、缓冲等）各个方面所消耗的空间，则都应计入。

另外，很多时候我们都是更多地甚至仅仅关注于算法的时间复杂度，而不必对空间复杂度做专门的考查。这种简便评测方式的依据，来自于以下事实：就渐进复杂度的意义而言，在任一算法的任何一次运行过程中所消耗的存储空间，都不会多于其间所执行基本操作的累计次数。

实际上根据定义，每次基本操作所涉及的存储空间，都不会超过常数规模；纵然每次基本操作所占用或访问的存储空间都是新开辟的，整个算法所需的空间总量，也不过与基本操作的次数同阶。从这个意义上说，时间复杂度本身就是空间复杂度的一个天然的上界。

当然，对空间复杂度的分析也有其自身的意义，尤其在对空间效率非常在乎的应用场合中，或当问题的输入规模极为庞大时，由时间复杂度所确立的平凡上界已经难以令人满意。这类情况下，人们将更为精细地考查不同算法的空间效率，并尽力在此方面不断优化。本书的后续章节，将结合一些实际问题介绍相关的方法与技巧。

§ 1.3 复杂度分析

在明确了算法复杂度的度量标准之后，如何分析具体算法的复杂度呢？1.2.2节所引入的三种记号中，大 O 记号是最基本的，也是最常用到的。从渐进分析的角度，大 O 记号将各算法的复杂度由低到高划分为若干层级级别。以下依次介绍若干典型的复杂度级别，并介绍主要的分析方法与技巧。读者可参照以下介绍的方法，做进一步的练习（习题[1-30]和[1-32]）。